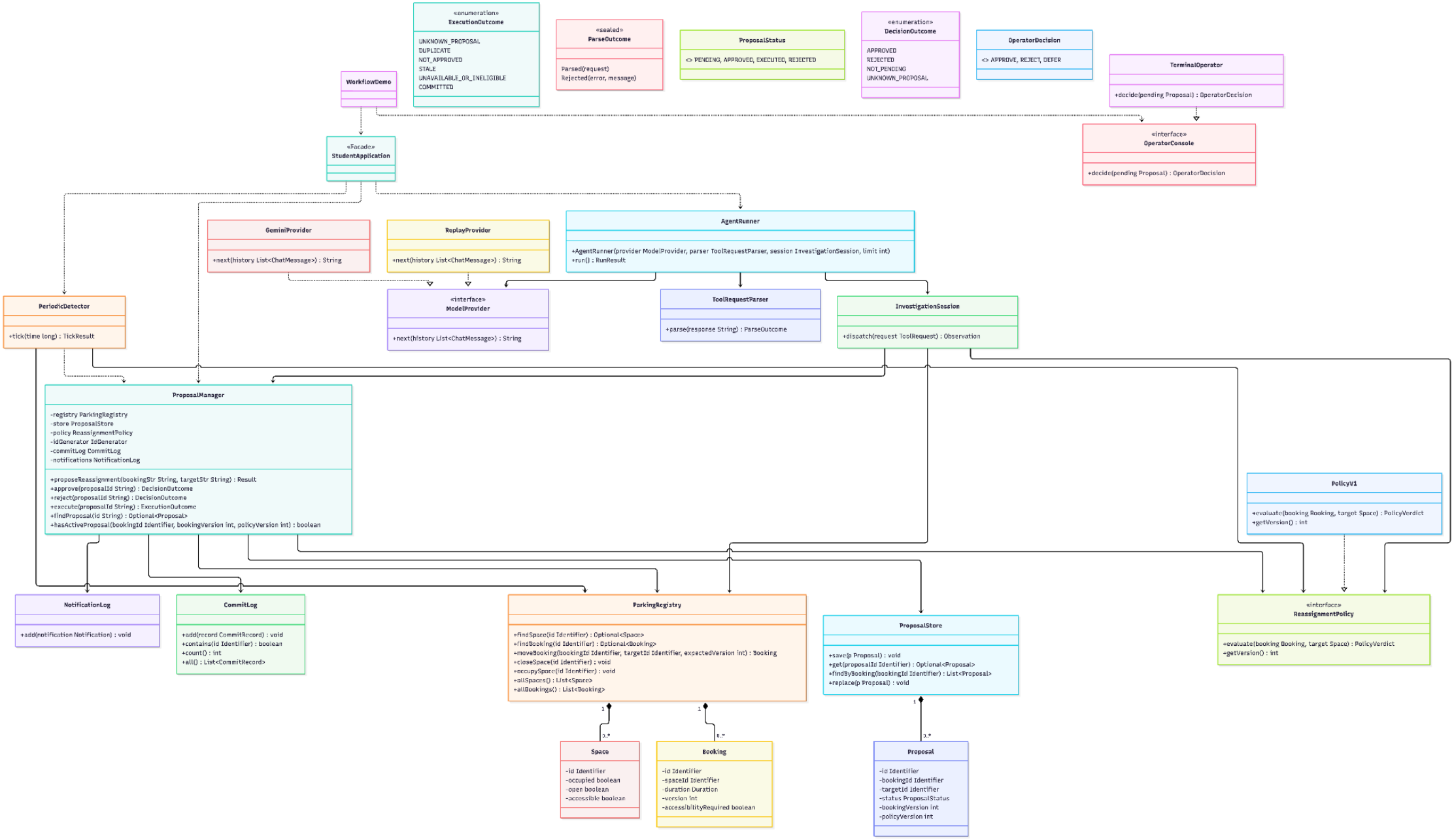
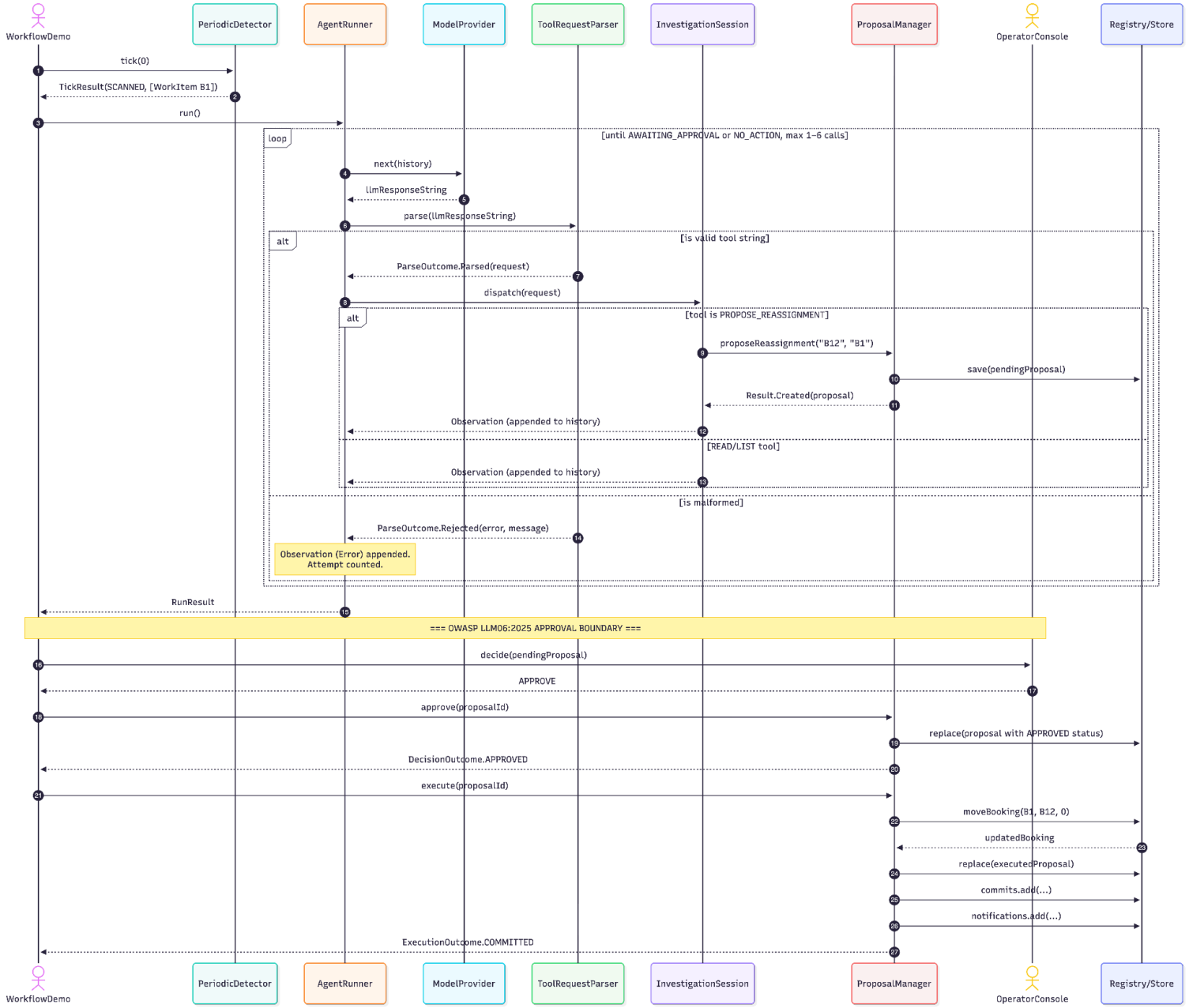
Michael Wiszniewski
D2 DESIGN.pdf
ParkOps
EECS 3311
Mark Atta Mensah

Updated class diagram:
What is new in D2 is the model protocol (ModelProvider, ToolRequestParser), the agent loop (AgentRunner, InvestigationSession), the periodic detection, the operator boundary, and two controlled mutation points ProposalStore.replace (status only) and ParkingRegistry.moveBooking (the commit step).

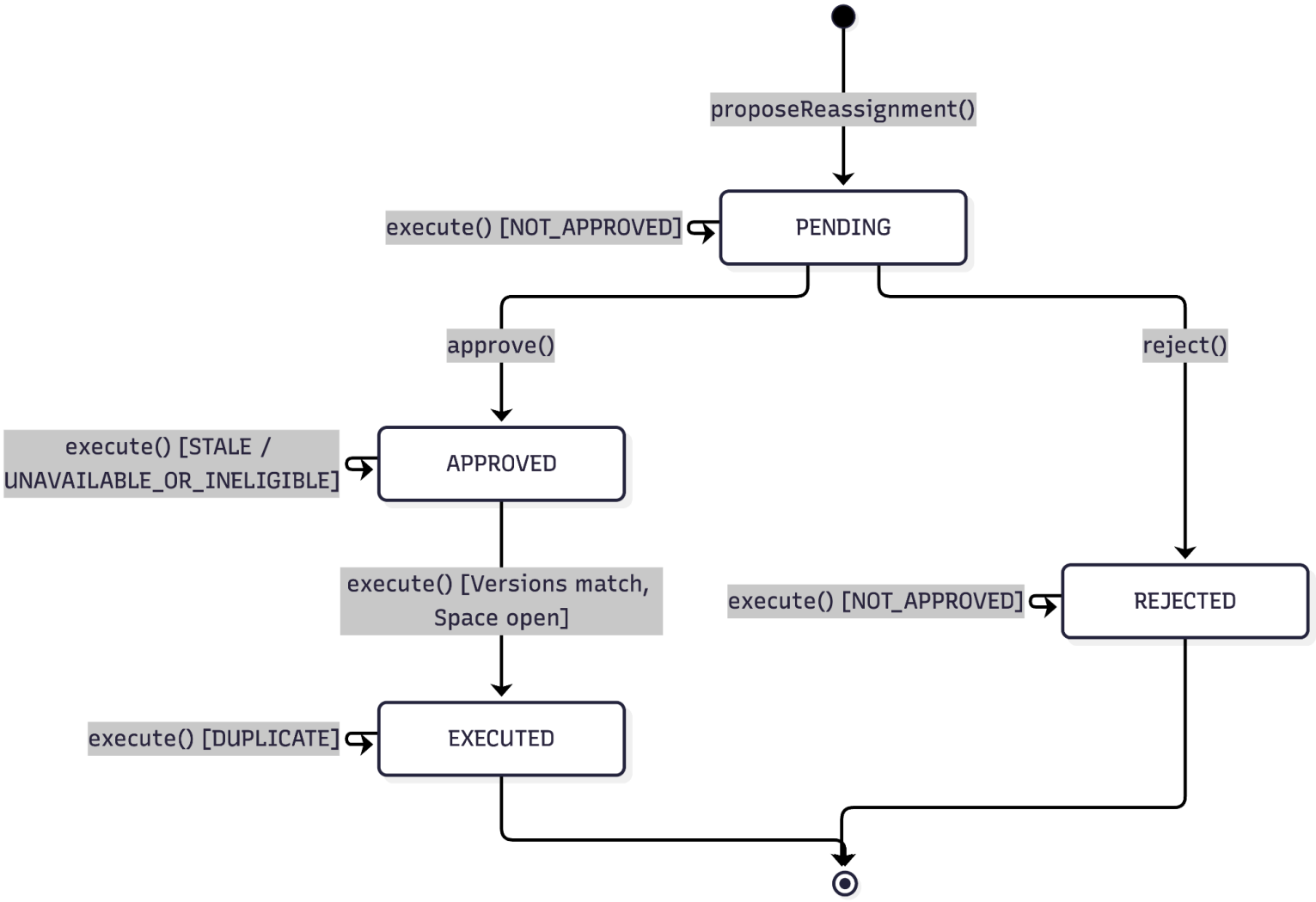

«enumeration»
ExecutionOutcome
UNKNOWN_PROPOSAL
DUPLICATE
NOT_APPROVED
STALE
UNAVAILABLE_OR_INELIGIBLE
COMMITTED
«sealed»
ParseOutcome
Parsed(request)
Rejected(error, message)
ProposalStatus
<> PENDING, APPROVED, EXECUTED, REJECTED
«enumeration»
DecisionOutcome
APPROVED
REJECTED
NOT_PENDING
UNKNOWN_PROPOSAL
OperatorDecision
<> APPROVE, REJECT, DEFER
TerminalOperator
+decide(pending Proposal) : OperatorDecision
WorkflowDemo
«interface»
OperatorConsole
+decide(pending Proposal) : OperatorDecision
«Facade»
StudentApplication
GeminiProvider
+next(history List<ChatMessage>) : String
ReplayProvider
+next(history List<ChatMessage>) : String
AgentRunner
+AgentRunner(provider ModelProvider, parser ToolRequestParser, session InvestigationSession, limit int)
+run() : RunResult
PeriodicDetector
+tick(time long) : TickResult
«interface»
ModelProvider
+next(history List<ChatMessage>) : String
ToolRequestParser
+parse(response String) : ParseOutcome
InvestigationSession
+dispatch(request ToolRequest) : Observation
ProposalManager
-registry ParkingRegistry
-store ProposalStore
-policy ReassignmentPolicy
-idGenerator IdGenerator
-commitLog CommitLog
-notifications NotificationLog
+proposeReassignment(bookingStr String, targetStr String) : Result
+approve(proposalId String) : DecisionOutcome
+reject(proposalId String) : DecisionOutcome
+execute(proposalId String) : ExecutionOutcome
+findProposal(id String) : Optional<Proposal>
+hasActiveProposal(bookingId Identifier, bookingVersion int, policyVersion int) : boolean
PolicyV1
+evaluate(booking Booking, target Space) : PolicyVerdict
+getVersion() : int
NotificationLog
+add(notification Notification) : void
CommitLog
+add(record CommitRecord) : void
+contains(id Identifier) : boolean
+count() : int
+all() : List<CommitRecord>
ParkingRegistry
+findSpace(id Identifier) : Optional<Space>
+findBooking(id Identifier) : Optional<Booking>
+moveBooking(bookingId Identifier, targetId Identifier, expectedVersion int) : Booking
+closeSpace(id Identifier) : void
+occupySpace(id Identifier) : void
+allSpaces() : List<Space>
+allBookings() : List<Booking>
ProposalStore
+save(p Proposal) : void
+get(proposalId Identifier) : Optional<Proposal>
+findByBooking(bookingId Identifier) : List<Proposal>
+replace(p Proposal) : void
«interface»
ReassignmentPolicy
+evaluate(booking Booking, target Space) : PolicyVerdict
+getVersion() : int
1
0..*
Space
-id Identifier
-occupied boolean
-open boolean
-accessible boolean
Booking
-id Identifier
-spaceId Identifier
-duration Duration
-version int
-accessibilityRequired boolean
Proposal
-id Identifier
-bookingId Identifier
-targetId Identifier
-status ProposalStatus
-bookingVersion int
-policyVersion int

Updated sequence diagram:

This diagram shows the looping of the agent which results in a PENDING proposal, then the human approval and boundary of execution.



Proposal state diagram:



Components of each layer of this design and their dependencies:

For the model protocol layer {ModelProvider (interface), GeminiProvider, ReplayProvider, ToolRequestParser}: The ModelProvider interface is implemented by GeminiProvider and ReplayProvider, and it is responsible for taking the next call in and returning untrusted response text. The model is only able to reach the four tool operations through the ToolRequestParser since there is no tool for approval, and ToolRequest has no approval field (this is to prevent the risks described in LLM06:2025 Excessive Agency). ToolRequestParser then turns that text into either a ToolRequest or a rejection. None of the classes in this layer touch the domain state, so when ReplayProvider is swapped in for GeminiProvider nothing changes.

For the application coordination layer {AgentRunner, InvestigationSession, PeriodicDetector, StudentApplication, WorkflowDemo, OperatorConsole}: The bounded loop and history is the responsibility of AgentRunner. InvestigationSession is responsible for enforcing the tool order (read, then list, then propose) and then rechecking the captured

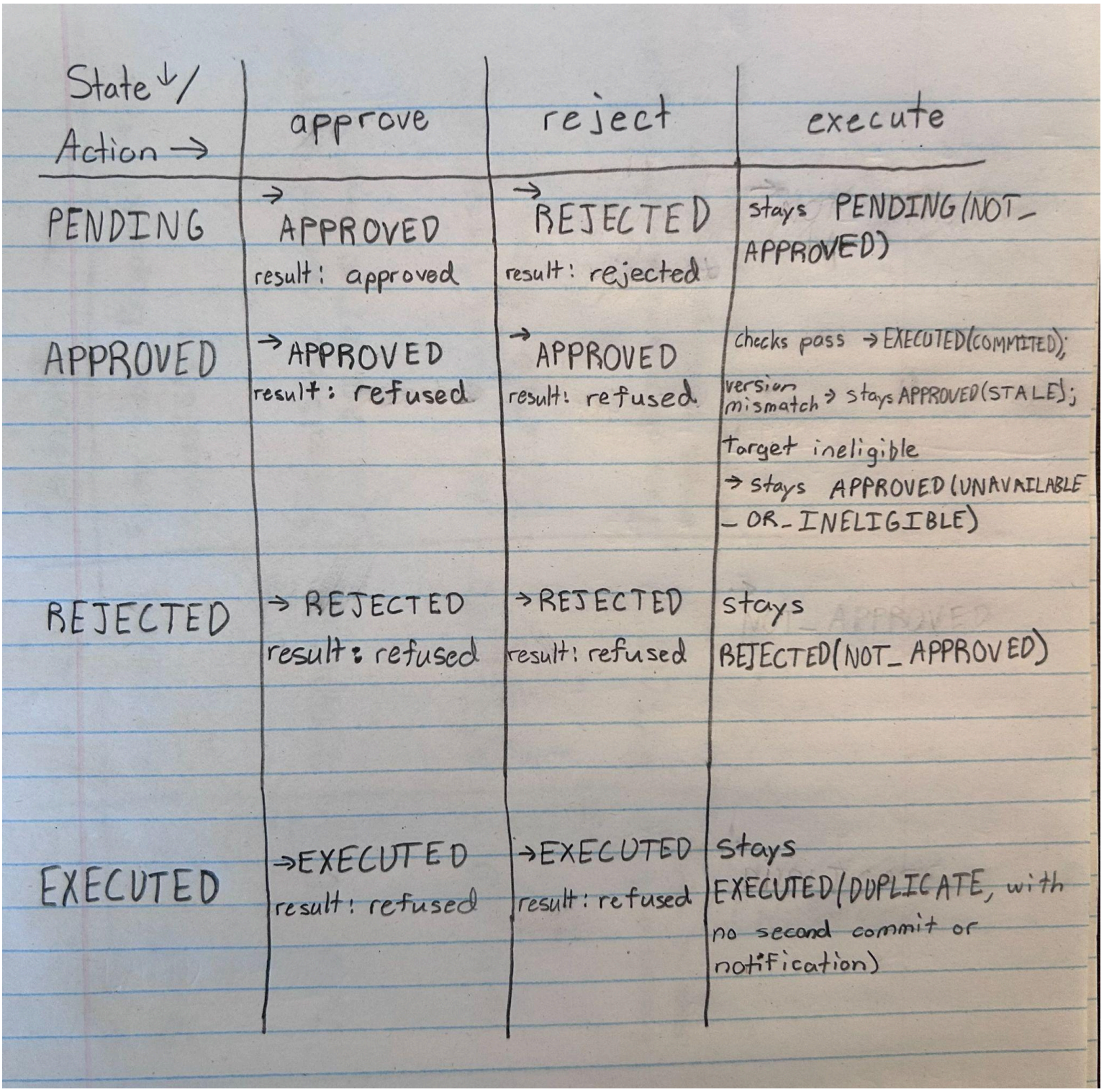
booking/policy versions before LIST and PROPOSE. The investigation restarts if the versions are stale. PeriodicDetector is responsible for the ticks and the retry policy detection only, it will never call the model itself or start an investigation. The OperatorConsole is the human decision which never gets seen by the agent. StudentApplication is the facade that is called on by each of the commands. The WorkflowDemo is the demo path that runs end to end and is shared by the live and replay commands.

For the domain rules layer {ProposalManager, ReassignmentPolicy/PolicyV1, entities}: The transition table and the execute checks are owned by ProposalManager. The policy decides if each proposal is eligible or not.

Data storage layer {ParkingRegistry, ProposalStore, CommitLog, NotificationLog}: Each class in this layer holds immutable snapshots and only gives out copies. Status is replaced by the ProposalStore.replace method (this method refuses to change anything else), and bookings are committed by the ParkingRegistry.moveBooking method (this method conducts checks on everything before it writes).

Dependencies: The application coordination layer is dependent on the protocol layer and the domain rules layer, the domain rules layer is dependent on the data storage layer, and the model protocol layer is not dependent on any of the other layers which is why swapping ReplayProvider in for GeminiProvider has no effect on any of the other components.

State transition table:



State ↓ / Action →	approve	reject	execute
PENDING	→ APPROVED result: approved	→ REJECTED result: rejected	stays PENDING (NOT_APPROVED)
APPROVED	→ APPROVED result: refused	→ APPROVED result: refused	checks pass → EXECUTED (COMMITED); version mismatch → stays APPROVED (STALE); target ineligible → stays APPROVED (UNAVAILABLE_OR_INELIGIBLE)
REJECTED	→ REJECTED result: refused	→ REJECTED result: refused	stays REJECTED (NOT_APPROVED)
EXECUTED	→ EXECUTED result: refused	→ EXECUTED result: refused	stays EXECUTED (DUPLICATE, with no second commit or notification)

Two design decisions and their alternatives:

One design decision I made was to keep Proposal strictly immutable. In order to change state, the application uses a store method that replaces the old Proposal with a newly created Proposal with the updated state change in effect, and the same ID. The obvious alternative here was to allow mutable state changes to be made directly on the Proposal objects. But the reason I chose to keep Proposal strictly immutable was because I felt it aligned closer to the

project outline requirement to utilize immutable snapshots. Keeping Proposal immutable prevents callers from bypassing controlled mutation.

Another design decision I made was to make it so that ProposalManager owns the state transition rules. This makes the service layer in charge of the state transitions. The ProposalManager checks the transition table (pictured above) to make sure that a proposal is currently PENDING, and then calls the ProposalStore method to make the swap. ProposalManager is the only class that holds the ProposalStore, and that replacement method will refuse to do anything other than changing the status. The alternative to this decision would've been to make it so the snapshot owns the rules. If this were the case, a method like proposal.approve() would evaluate its own state and return a new Proposal (or refuse if not PENDING), and then the ProposalStore would just accept the resulting object. The reason I chose to have it this way is because I wanted the business logic separate from the data models. I wanted ProposalManager to be the centre of the service layer, in charge of orchestrating the transition table and then calling the ProposalStore methods. This allows for the snapshots to be pure data records, while the rules are centralized in the service layer. I believe this to be the more intuitive approach.

One rejected model/tool interaction:



Shown above is a live Gemini run on 2026-11-08 at 14:12:39 (results/live-run-20261108-141239.txt). In this run, calls 1 and 2 succeeded, and call 3 failed at the provider. The system handled it by returning PROVIDER_ERROR after counting the 3 calls. There was no retry automatically, also no proposal created. The operator was therefore not prompted anything, nothing was committed, meaning the booking was not changed. This failure occurred in the middle of an investigation after READ and LIST were already successful. Despite this, no partial state was left over as a result.

Limitations:

Not ACID compliant: Currently, my architecture models sequential ticks and decisions, but there is a lack of ACID compliance (Atomicity, Consistency, Isolation, and Durability). If this were an actual application in a real deployed environment, I would need to have

database-level row locks in execute() or some sort of concurrency control on the Booking version in order to prevent two operators from executing overlapping proposals at the same time and causing unintended effects. Therefore in a real world environment, my system would have no guarantee of reliable and accurate transaction processing in the event of errors in the system or power failures.

No authentication or role-based access control: My OperatorConsole trusts every human to make decisions. If this were a real system, it would require Role-Based Access Control (RBAC) to manage what the user can do, and authentication (OIDC for example) to prove that the user is who they say they are. Without these, we cannot securely verify who approved or rejected a proposal. In a real world system, we would need to cryptographically bind the identity of the operator that accepted or rejected each proposal to the CommitLog, in order to meet the audit and compliance standards that would likely be in place.

Lack of error detail records: Upon unsuccessful or rejected model/tool interactions, the details of provider errors are not kept in the results transcript, so we have no record of the cause of the error.

Others:

- An attempt is wasted because code-fenced replies are simply rejected instead of being cleaned up.
- Because the project uses Javas default package, replace can’t be hidden by access modifiers so protection comes from reference ownership